

Implement DDL operations

8 minutes

Your data engineering team needs to manage complex data transformations, enforce data quality rules, and maintain streaming pipelines across multiple environments. You need **reusable functions** to standardize calculations, **stored procedures** to orchestrate multi-step operations, and **streaming tables** to process real-time data. Unity Catalog's **Data Definition Language (DDL) operations** provide the tools to create and modify these objects while maintaining governance and version control.

Create functions for reusable logic

Functions encapsulate business logic that you can reuse across queries and transformations. Unity Catalog supports both **scalar functions** that return single values and **table-valued functions** that return entire result sets. You can write functions in **SQL** or **Python**, depending on your team's expertise and the complexity of the logic.

When you create a function in Unity Catalog, it becomes a governed object that you can share across teams and workspaces. With Unity Catalog, you define functions once and invoke them anywhere within your data platform. The catalog tracks function ownership, dependencies, and usage patterns, providing visibility into how your organization uses shared logic.

SQL scalar functions work well for simple calculations and transformations. You define the function with parameters, specify the return type, and provide the SQL expression that computes the result:

SQL

```
CREATE FUNCTION dev_catalog.finance_schema.calculate_discount(price DOUBLE,  
discount_rate DOUBLE)  
RETURNS DOUBLE
```

```
COMMENT 'Calculate discounted price'  
RETURN price * (1 - discount_rate);
```

This function accepts two numeric parameters and returns the discounted price. You can now call this function from any query within your Unity Catalog environment, ensuring consistent discount calculations across all reports and analytics.

Python functions handle more complex logic that requires external libraries or advanced data processing. Unity Catalog executes Python functions using serverless or pro SQL warehouses:

SQL

```
CREATE FUNCTION dev_catalog.analytics_schema.normalize_text(input_text STRING)  
RETURNS STRING  
LANGUAGE PYTHON  
COMMENT 'Normalize text using Python'  
AS $$  
    return input_text.lower().strip() if input_text else None  
$$;
```

Table-valued functions return complete result sets rather than scalar values. This approach works well when you need to transform arrays into rows or generate multiple records from a single input:

SQL

```
CREATE FUNCTION dev_catalog.utils_schema.split_to_rows(input_array  
ARRAY<STRING>)  
RETURNS TABLE(value STRING)  
RETURN SELECT explode(input_array) AS value;
```

You can then query this function like a table: `SELECT * FROM dev_catalog.utils_schema.split_to_rows(array('a', 'b', 'c'))`.

You can also create table-valued functions using **Python**, which is useful when you need more complex processing logic:

SQL

```
CREATE FUNCTION dev_catalog.utils_schema.generate_sequence(start INT, end INT)  
RETURNS TABLE(num INT)  
LANGUAGE PYTHON  
COMMENT 'Generate a sequence of numbers from start to end'
```

```
AS $$  
  return [(i,) for i in range(start, end + 1)]  
$$;
```

Temporary functions exist only within your current session and don't persist in the catalog. This approach works well for one-time analyses or testing new logic before creating a permanent function:

SQL

```
CREATE TEMPORARY FUNCTION calculate_tax(amount DOUBLE)  
RETURNS DOUBLE  
RETURN amount * 0.08;
```

Temporary functions don't require catalog or schema qualifiers because they exist only in your session scope.

Create procedures for multi-step operations

SQL stored procedures let you define sequences of SQL statements that run together, with support for variables, control flow, and error handling. Procedures become particularly valuable when you need to orchestrate complex data operations that involve multiple steps, conditional logic, or iterative processing.

With procedures, you can encapsulate entire workflows—like data validation, transformation, and loading—into a single callable unit. You create procedures using the `CREATE PROCEDURE` statement, which requires Unity Catalog and Databricks Runtime 17.0 or above:

SQL

```
CREATE PROCEDURE dev_catalog.etl_schema.refresh_customer_summary(  
  IN source_date DATE,  
  OUT rows_processed INT  
)  
LANGUAGE SQL  
SQL SECURITY INVOKER  
COMMENT 'Refresh customer summary table for specified date'  
BEGIN  
  DECLARE row_count INT;  
  
  DELETE FROM dev_catalog.analytics_schema.customer_summary  
  WHERE summary_date = source_date;
```

```
INSERT INTO dev_catalog.analytics_schema.customer_summary
SELECT customer_id, summary_date, SUM(amount) as total
FROM dev_catalog.sales_schema.transactions
WHERE transaction_date = source_date
GROUP BY customer_id, summary_date;

SET row_count = (SELECT COUNT(*) FROM dev_catalog.analytics_schema.customer_summary
                 WHERE summary_date = source_date);
SET rows_processed = row_count;
END;
```

This procedure accepts an input date parameter and returns the number of rows processed through an output parameter. You call procedures using the `CALL` statement:

SQL

```
CALL dev_catalog.etl_schema.refresh_customer_summary(DATE '2024-01-15',
:rows);
```

Procedures support **IN** parameters for inputs, **OUT** parameters for outputs, and **INOUT** parameters that serve both purposes. The `SQL SECURITY INVOKER` clause specifies that the procedure runs with the permissions of the user calling it, ensuring proper access control.

Modify catalog ownership and properties

As your data organization evolves, you need to transfer ownership of catalogs or modify their properties. The `ALTER CATALOG` statement provides these capabilities, letting you change catalog owners, apply tags for metadata management, or enable predictive optimization.

Transferring ownership ensures that the right team or principal manages each catalog. You might transfer ownership when reorganizing teams or delegating catalog management to a different group:

SQL

```
ALTER CATALOG dev_catalog SET OWNER TO `data_engineering_team`;
```

This command transfers ownership to a group named `data_engineering_team`. Only the current owner, a metastore admin, or a user with the `MANAGE` privilege can transfer ownership.

Applying tags helps you organize and classify catalogs according to your organization's taxonomy. Tags provide searchable metadata that supports governance and discovery:

SQL

```
ALTER CATALOG dev_catalog SET TAGS (  
  environment = 'development',  
  cost_center = 'engineering',  
  data_classification = 'internal'  
);
```

Enabling predictive optimization automates maintenance tasks like compaction and vacuum operations. Unity Catalog monitors usage patterns and applies optimizations when appropriate:

SQL

```
ALTER CATALOG prod_catalog ENABLE PREDICTIVE OPTIMIZATION;
```

Tables within the catalog inherit this setting unless you explicitly override it at the table level.

Modify table structure and constraints

As your data requirements change, you need to modify table schemas without rebuilding entire datasets. The `ALTER TABLE` statement supports adding columns, renaming columns, dropping columns, managing constraints, and changing column types—all while preserving your existing data.

Adding columns extends your table schema to accommodate new data fields. When you add a column to an existing Delta table, Unity Catalog populates it with `NULL` values for existing rows:

SQL

```
ALTER TABLE dev_catalog.sales_schema.orders  
ADD COLUMN shipping_carrier STRING  
COMMENT 'Name of the shipping carrier';
```

Renaming columns updates column names without moving data. This operation requires Delta Lake column mapping, which you enable by setting table properties:

SQL

```
ALTER TABLE dev_catalog.sales_schema.orders  
RENAME COLUMN customer_email TO email_address;
```

Dropping columns removes columns from your table schema. You must drop any dependent constraints or generated columns first:

SQL

```
ALTER TABLE dev_catalog.sales_schema.orders  
DROP COLUMN temporary_field;
```

Changing column types adapts your schema to new requirements. You can widen types (like `INT` to `BIGINT`) or change compatible types while Unity Catalog handles data conversion:

SQL

```
ALTER TABLE dev_catalog.sales_schema.orders  
ALTER COLUMN quantity TYPE BIGINT;
```

These operations work on Unity Catalog tables and require the `MODIFY` privilege.

Next unit: Implement foreign catalog

[< Previous](#)[Next >](#)